# VisitorQR

## Visitor Management System

# REST API Documentation

Version 2.2 | FastAPI + SQLAlchemy + SQLite

`Base URL: http://localhost:8000/api`

**Authentication: JWT Bearer Token**
Interactive Docs: http://localhost:8000/docs
Roles: guard | admin | super_admin

| Field | Details |
|---|---|
| **Document Type** | REST API Documentation |
| **System Name** | QR-Based Visitor Management System |
| **Version** | v2.0.0 |
| **Prepared By** | Amit Kushwaha |
| **Department** | MP Online Limited |
| **Date** | 27 April 2026 |
| **Project Type** | Internship Project |

# 1. Overview

The VisitorQR API is a RESTful backend for a QR-based Visitor Management System. It enables visitor self-registration, QR code generation and delivery, smart check-in/check-out via QR scan, and full administrative control over users, branches, and audit logs.

## 1.1 Base URL & Format

```
Base URL : http://localhost:8000/api
Format   : All requests and responses use JSON (application/json)
Auth     : Bearer JWT token in Authorization header
Docs     : http://localhost:8000/docs  (Swagger UI)
```

## 1.2 User Roles

All protected endpoints enforce role-based access control (RBAC). Three roles exist:

| Role | Access Level | Capabilities |
|---|---|---|
| `guard` | Limited | Scan QR codes to check visitors in and out. View visit list. |
| `admin` | Standard | All guard permissions + Dashboard, Reports, Visitor management, Branch listing. |
| `super_admin` | Full | All admin permissions + User management, Branch creation, Audit log access. |

## 1.3 Authentication

Obtain a token by calling POST /auth/login with form data (username + password). Include the token in all protected requests:

```
Authorization: Bearer <your_jwt_token>
```

Tokens expire after 24 hours. A 401 response means the token is missing, invalid, or expired.

## 1.4 HTTP Status Codes

| Code | Meaning |
|---|---|
| `200 OK` | Request succeeded. Response body contains the result. |
| `201 Created` | Resource created successfully. |

| Code | Meaning |
|---|---|
| `204 No Content` | Success with no response body (e.g. delete). |
| `400 Bad Request` | Invalid input data or validation error. |
| `401 Unauthorized` | Missing or invalid JWT token. |
| `403 Forbidden` | Authenticated but insufficient role. |
| `404 Not Found` | Requested resource does not exist. |
| `422 Unprocessable` | Request body failed schema validation (Pydantic). |
| `500 Server Error` | Unexpected server-side error. |

# 2. Endpoint Summary

All endpoints are prefixed with /api. The table below provides a quick reference of every available route.

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | `/auth/login` | Login and receive a JWT token | *Public* |
| GET | `/auth/me` | Get the logged-in user profile | *Any role* |
| PUT | `/auth/profile` | Update own name / phone / department | *Any role* |
| POST | `/auth/change-password` | Change own password | *Any role* |
| POST | `/visitors/register` | Register a visitor and generate QR pass | *Public* |
| GET | `/visitors` | List all visitors | *guard+* |
| GET | `/visitors/{visitor_id}` | Get a single visitor by ID | *guard+* |
| POST | `/visits/scan/{qr_token}` | Scan QR to check-in or check-out | *guard+* |
| GET | `/visits` | List all visits (filter by branch/status) | *guard+* |
| GET | `/visits/{visit_id}` | Get a single visit by ID | *guard+* |
| GET | `/admin/users` | List all system users | *super_admin* |
| POST | `/admin/users` | Create a new system user | *super_admin* |
| DELETE | `/admin/users/{user_id}` | Deactivate a user account | *super_admin* |
| GET | `/admin/branches` | List all active office branches | *Public* |
| POST | `/admin/branches` | Create a new branch | *super_admin* |
| GET | `/admin/dashboard` | Get real-time dashboard statistics | *admin+* |
| GET | `/admin/report` | Get visit analytics report | *admin+* |
| GET | `/admin/audit-logs` | List all audit log entries | *super_admin* |
| GET | `/admin/audit-logs/search` | Search audit logs with filters | *super_admin* |

# 3. Authentication Endpoints

Auth endpoints handle login, token refresh, profile management, and password changes.

## 3.1 POST /auth/login

**Description**
Authenticate with email and password. Returns a JWT Bearer token valid for 24 hours. Uses OAuth2 form data format (not JSON).

### Request (form-urlencoded)

| Field | Type | Required | Description |
|---|---|---|---|
| **username** | string (email) | Yes | The user's email address (used as username) |
| **password** | string | Yes | The user's plaintext password |

### Response — 200 OK

| Field | Type | Required | Description |
|---|---|---|---|
| **access_token** | string | — | JWT Bearer token. Include in Authorization header. |
| **token_type** | string | — | Always 'bearer' |
| **user_id** | integer | — | Unique user ID |
| **user_name** | string | — | Full name of the logged-in user |
| **user_role** | string | — | Role: guard \| admin \| super_admin |
| **branch_id** | integer\|null | — | Branch the user belongs to (null if unassigned) |

### Example Request

```
POST /api/auth/login
Content-Type: application/x-www-form-urlencoded

username=admin@company.com&password=secret123
```

### Example Response

```
{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer", "user_id": 1,
  "user_name": "System Admin", "user_role": "super_admin", "branch_id": null }
```

## 3.2 GET /auth/me

**Description**
Returns the full profile of the currently authenticated user. No request body needed.

### Response — 200 OK

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | integer | — | User's unique ID |
| `full_name` | string | — | Full name |
| `email` | string | — | Email address (used to login) |
| `phone` | string\|null | — | Optional phone number |
| `department` | string\|null | — | Optional department name |
| `role` | string | — | guard \| admin \| super_admin |
| `branch_id` | integer\|null | — | Branch ID or null |
| `is_active` | boolean | — | Whether the account is active |
| `created_at` | datetime | — | Account creation timestamp (UTC ISO 8601) |

## 3.3 PUT /auth/profile

**Description**
Update the logged-in user's own display name, phone number, or department. Role and branch can only be changed by a super_admin.

### Request Body (JSON)

| Field | Type | Required | Description |
|---|---|---|---|
| `full_name` | string | No | New display name (all fields optional) |
| `phone` | string | No | New phone number |
| `department` | string | No | New department name |

## 3.4 POST /auth/change-password

**Description**
Change the authenticated user's own password. The current password is required for verification.

### Request Body (JSON)

| Field | Type | Required | Description |
|---|---|---|---|
| `current_password` | `string` | Yes | The user's existing password for verification |
| `new_password` | `string` | Yes | The desired new password (min 8 characters recommended) |

# 4. Visitor Registration Endpoints

These endpoints handle the core visitor lifecycle — registration and QR pass generation.

## 4.1 POST/visitors/register

**Description**
PUBLIC endpoint — no authentication needed. Registers a new visitor, creates a visit record, generates a unique QR code, and optionally emails the QR pass to the visitor. Admins and super_admins can additionally set custom QR validity duration.

### Request Body (JSON)

| Field | Type | Required | Description |
|---|---|---|---|
| full_name | string | Yes | Visitor's full name |
| phone | string | Yes | Contact phone number |
| email | string | No | Email address — QR code will be emailed here if SMTP is configured |
| company_name | string | No | Visitor's company or organization |
| host_name | string | No | Name of the employee being visited |
| purpose | string | No | Visit purpose: meeting \| delivery \| interview \| maintenance \| other (default: meeting) |
| branch_id | integer | Yes | ID of the office branch being visited |
| notes | string | No | Additional notes or comments |
| photo_data | string | No | Base64 data URI of visitor photo (e.g. '...') |
| qr_validity_type | string | No | Admin only: one_time \| hourly \| daily \| weekly \| monthly (default: uses .env QR_EXPIRY_HOURS) |
| qr_hours | integer | No | Admin only: number of hours (1–8760), used only when qr_validity_type = hourly |

### Response — 201 Created

| Field | Type | Required | Description |
|---|---|---|---|
| id | integer | — | Visit record ID |
| visitor_id | integer | — | Visitor record ID |
| branch_id | integer | — | Branch where visit is registered |
| host_name | string\|null | — | Person being visited |

| Field | Type | Required | Description |
|---|---|---|---|
| purpose | string | — | Visit purpose value |
| qr_token | string | — | Unique token embedded in the QR code |
| qr_image | string | — | Base64 data URI of the QR code PNG — use as <img src={qr_image}> in frontend |
| qr_expires | datetime | — | QR code expiry timestamp (UTC ISO 8601) |
| qr_expiry_hours | integer\|null | — | Hours until QR expires |
| qr_validity_type | string | — | hourly \| daily \| weekly \| monthly \| one_time |
| status | string | — | registered \| checked_in \| checked_out \| expired |
| registered_at | datetime | — | Registration timestamp (UTC ISO 8601) |
| visitor | object | — | Nested visitor object: id, full_name, phone, email, company_name, photo_path |
| email_sent | boolean | — | True if QR email was sent successfully |
| email_address | string | — | Email address the QR was sent to (empty string if not sent) |

### QR Validity Options (Admin Only)

| qr_validity_type | Duration | Behaviour |
|---|---|---|
| (not set) | From .env | Uses QR_EXPIRY_HOURS value set in backend .env file |
| one_time | Variable | QR becomes invalid immediately after the visitor checks out |
| hourly | 1–8760 hrs | Custom hours specified in qr_hours field |
| daily | 24 hours | QR is valid for exactly 24 hours from registration |
| weekly | 7 days | QR is valid for 168 hours (7 days) from registration |
| monthly | 30 days | QR is valid for 720 hours (30 days) from registration |

## 4.2 GET /visitors

**Description**
Returns a paginated list of all registered visitors. Requires guard role or higher.

### Query Parameters

| Field | Type | Required | Description |
|---|---|---|---|
| skip | integer | No | Number of records to skip (default: 0) |

| Field | Type | Required | Description |
|---|---|---|---|
| `limit` | integer | No | Maximum records to return (default: 50, max: 200) |

## 4.3 GET /visitors/{visitor_id}

**Description**
Returns the profile of a single visitor by their ID. Returns 404 if not found.

### Path Parameter

| Field | Type | Required | Description |
|---|---|---|---|
| `visitor_id` | integer | Yes | The unique ID of the visitor to retrieve |

# 5. Visit & Scan Endpoints

These endpoints handle the physical check-in and check-out process at the office entrance via QR code scanning.

## 5.1 POST /visits/scan/{qr_token}

**Description**
Smart scan endpoint. The guard scans the visitor's QR code. The system automatically determines the action: - If the visitor has not checked in yet → performs Check-In - If the visitor is already checked in → performs Check-Out The scanning guard's user ID is logged for audit purposes.

### Path Parameter

| Field | Type | Required | Description |
|---|---|---|---|
| qr_token | string | Yes | The unique token extracted from the visitor's QR code |

### Response — 200 OK

| Field | Type | Required | Description |
|---|---|---|---|
| message | string | — | Human-readable result: 'Checked In' or 'Checked Out' |
| visit_id | integer | — | The visit record ID |
| visitor_name | string | — | Full name of the visitor |
| visitor_phone | string | — | Visitor's phone number |
| host_name | string | — | Name of the host being visited |
| purpose | string | — | Visit purpose |
| action | string | — | check_in or check_out |
| timestamp | datetime | — | When the scan occurred (UTC ISO 8601) |

### Error Responses

| Field | Type | Required | Description |
|---|---|---|---|
| 404 Not Found | – | — | No visit found with the provided QR token |
| 409 Conflict | – | — | QR code has expired or visitor is already checked out |
| 401 Unauthorized | – | — | Guard is not logged in |
| 403 Forbidden | – | — | Authenticated user does not have scan permission |

## 5.2 GET /visits

**Description**
Returns a list of visit records. Supports filtering by branch and status.

### Query Parameters

| Field | Type | Required | Description |
|---|---|---|---|
| `branch_id` | integer | No | Filter visits by branch ID |
| `status` | string | No | Filter by status: registered \| checked_in \| checked_out \| expired |
| `skip` | integer | No | Pagination offset (default: 0) |
| `limit` | integer | No | Max results (default: 50, max: 200) |

## 5.3 GET /visits/{visit_id}

**Description**
Returns full details of a single visit record including nested visitor information.

### Path Parameter

| Field | Type | Required | Description |
|---|---|---|---|
| `visit_id` | integer | Yes | The unique ID of the visit record |

# 6. Admin Endpoints

Admin endpoints are for user management, branch management, dashboard statistics, and reporting. All require admin or super_admin role unless noted.

## 6.1 GET /admin/users

**Description**
Returns a paginated list of all registered system users (guards, admins, super_admins). Requires super_admin.

### Query Parameters

| Field | Type | Required | Description |
|---|---|---|---|
| skip | integer | No | Pagination offset (default: 0) |
| limit | integer | No | Max results (default: 100, max: 500) |

## 6.2 POST /admin/users

**Description**
Creates a new system user account. The password is hashed before storage. Requires super_admin.

### Request Body (JSON)

| Field | Type | Required | Description |
|---|---|---|---|
| full_name | string | Yes | Full name of the new user |
| email | string | Yes | Email address (used as login username, must be unique) |
| password | string | Yes | Initial password (will be bcrypt hashed) |
| phone | string | No | Optional phone number |
| department | string | No | Optional department name |
| role | string | No | guard \| admin \| super_admin (default: guard) |
| branch_id | integer | No | Branch the user will be assigned to |

## 6.3 DELETE /admin/users/{user_id}

**Description**

Deactivates a user account (soft delete — sets is_active=false). The user can no longer log in but their records are preserved for audit purposes. Requires super_admin.

### Path Parameter

| Field | Type | Required | Description |
|---|---|---|---|
| user_id | integer | Yes | The ID of the user to deactivate |

## 6.4 GET /admin/branches

**Description**
Returns a list of all active office branches. This endpoint is PUBLIC — no authentication required. Used by the visitor registration form to populate the branch dropdown.

### Response Fields

| Field | Type | Required | Description |
|---|---|---|---|
| id | integer | — | Unique branch ID |
| name | string | — | Branch name (e.g. 'Mumbai HQ') |
| city | string\|null | — | City where the branch is located |
| address | string\|null | — | Full address |
| is_active | boolean | — | Whether the branch is currently active |
| created_at | datetime | — | When the branch was created (UTC ISO 8601) |

## 6.5 POST /admin/branches

**Description**
Creates a new office branch. Requires super_admin.

### Request Body (JSON)

| Field | Type | Required | Description |
|---|---|---|---|
| name | string | Yes | Branch name (must be unique and descriptive) |
| city | string | No | City of the branch |
| address | string | No | Full postal address |

## 6.6 GET /admin/dashboard

> **Description**
> Returns real-time statistics for the office dashboard. Requires admin or super_admin.

### Query Parameters

| Field | Type | Required | Description |
|---|---|---|---|
| `branch_id` | integer | No | Filter dashboard stats by a specific branch (null = all branches) |

### Response Fields

| Field | Type | Required | Description |
|---|---|---|---|
| `currently_inside` | integer | — | Number of visitors currently checked in |
| `total_today` | integer | — | Total visits registered today |
| `total_this_week` | integer | — | Total visits registered this calendar week |
| `avg_duration_mins` | float | — | Average visit duration in minutes |
| `purpose_breakdown` | object | — | Count of visits per purpose type |
| `currently_inside_list` | array | — | List of visitor details for those currently inside |
| `recent_activity` | array | — | Last 20 visit events with timestamps |
| `generated_at` | string | — | Timestamp when this data was generated |

## 6.7 GET /admin/report

> **Description**
> Returns historical visit analytics for a configurable time period. Requires admin or super_admin.

### Query Parameters

| Field | Type | Required | Description |
|---|---|---|---|
| `branch_id` | integer | No | Filter by branch ID (null = all branches) |
| `days` | integer | No | Report period in days (min: 7, max: 365, default: 30) |

### Response Fields

| Field | Type | Required | Description |
|---|---|---|---|
| `period_days` | integer | — | The number of days the report covers |
| `daily_counts` | array | — | List of {date, count} objects for each day in the period |
| `top_hosts` | array | — | Top 10 most visited hosts with visit counts |
| `total_visits` | integer | — | Total visits in the period |

# 7. Audit Log Endpoints

Audit logs record every significant action taken in the system — logins, registrations, check-ins, user changes, etc. Access requires super_admin role.

## 7.1 GET /admin/audit-logs

**Description**
Returns the most recent audit log entries in reverse chronological order. Requires super_admin.

### Query Parameters

| Field | Type | Required | Description |
|---|---|---|---|
| skip | integer | No | Pagination offset (default: 0) |
| limit | integer | No | Max results (default: 100, max: 500) |

### Response — Array of log objects

| Field | Type | Required | Description |
|---|---|---|---|
| id | integer | — | Unique log entry ID |
| user_id | integer\|null | — | ID of the user who performed the action (null for system actions) |
| action | string | — | Action name: user_login \| visitor_registered \| visitor_checkin \| visitor_checkout \| user_created \| user_deactivated \| profile_updated |
| entity | string\|null | — | Type of record affected: visit \| visitor \| user \| branch |
| entity_id | integer\|null | — | ID of the affected record |
| details | string\|null | — | Additional context as JSON string or plain text |
| ip_address | string\|null | — | IP address of the client that performed the action |
| created_at | datetime | — | When the action occurred (UTC ISO 8601) |

## 7.2 GET /admin/audit-logs/search

**Description**
Advanced filtered search across audit logs. All filter parameters are optional and can be combined freely. Requires super_admin.

## Query Parameters

| Field | Type | Required | Description |
|---|---|---|---|
| **user_id** | integer | No | Filter logs by a specific user's ID |
| **action** | string | No | Partial match on action name (e.g. 'login' matches 'user_login') |
| **entity** | string | No | Filter by entity type: visit \| visitor \| user \| branch |
| **entity_id** | integer | No | Filter by a specific entity's ID |
| **date** | string | No | Exact date filter (YYYY-MM-DD format) — returns logs for that day only |
| **date_from** | string | No | Start of date range (YYYY-MM-DD). Combine with date_to for a range. |
| **date_to** | string | No | End of date range (YYYY-MM-DD, inclusive up to 23:59:59) |
| **last_n_days** | integer | No | Shortcut: return logs from the last N days (e.g. 7 for last week) |
| **time_from** | string | No | Filter by time of day from (HH:MM format, applies to created_at) |
| **time_to** | string | No | Filter by time of day to (HH:MM format) |
| **ip_address** | string | No | Partial match on IP address (e.g. '192.168' matches any 192.168.x.x) |
| **sort_order** | string | No | asc for oldest first, desc for newest first (default: desc) |
| **skip** | integer | No | Pagination offset (default: 0) |
| **limit** | integer | No | Max results (default: 50, max: 500) |

## Response

| Field | Type | Required | Description |
|---|---|---|---|
| **total** | integer | — | Total number of matching records (before pagination) |
| **skip** | integer | — | Offset that was applied |
| **limit** | integer | — | Limit that was applied |
| **results** | array | — | Array of matching audit log objects (same fields as GET /admin/audit-logs) |

## Example Search Requests

```
GET /api/admin/audit-logs/search?action=login&last_n_days=7
GET /api/admin/audit-logs/search?date=2026-04-24&entity=visit
```

```
GET /api/admin/audit-logs/search?user_id=3&date_from=2026-04-01&date_to=2026-04-30
GET /api/admin/audit-logs/search?time_from=09:00&time_to=17:00&sort_order=asc
```

# 8. Data Models

The following tables describe the database schema used by the VMS backend.

## 8.1 users

| Column | Type | Description |
|---|---|---|
| id | INTEGER PK | Auto-increment primary key |
| full_name | VARCHAR(100) | User's full name |
| email | VARCHAR(150) | Unique email address (login username) |
| hashed_password | VARCHAR(255) | bcrypt-hashed password |
| phone | VARCHAR(20) | Optional phone number |
| department | VARCHAR(80) | Optional department |
| role | ENUM | guard \| admin \| super_admin |
| branch_id | INTEGER FK | References branches.id (nullable) |
| is_active | BOOLEAN | False = deactivated, cannot login |
| created_at | DATETIME | Account creation timestamp (UTC) |
| last_login | DATETIME | Last successful login timestamp (UTC, nullable) |

## 8.2 visitors

| Column | Type | Description |
|---|---|---|
| id | INTEGER PK | Auto-increment primary key |
| full_name | VARCHAR(100) | Visitor's full name |
| phone | VARCHAR(20) | Phone number |
| email | VARCHAR(150) | Optional email address |
| company_name | VARCHAR(120) | Optional company or organization |
| photo_path | VARCHAR(255) | Relative path to visitor photo in uploads/photos/ |
| created_at | DATETIME | First registration timestamp (UTC) |

## 8.3 visits

| Column | Type | Description |
|---|---|---|
| id | INTEGER PK | Auto-increment primary key |

| Column | Type | Description |
|---|---|---|
| visitor_id | INTEGER FK | References visitors.id |
| branch_id | INTEGER FK | References branches.id |
| host_name | VARCHAR(100) | Name of employee being visited |
| purpose | ENUM | meeting \| delivery \| interview \| maintenance \| other |
| notes | TEXT | Optional additional notes |
| qr_token | VARCHAR(64) | Unique token embedded in the QR code |
| qr_image | TEXT | Base64 PNG data URI of the generated QR code |
| qr_expires | DATETIME | QR expiry timestamp (UTC) |
| qr_expiry_hours | INTEGER | Hours until expiry (stored for audit reference) |
| status | ENUM | registered \| checked_in \| checked_out \| expired |
| scanned_by | INTEGER FK | User ID of guard who scanned (nullable) |
| checked_in_at | DATETIME | Check-in timestamp (UTC, nullable) |
| checked_out_at | DATETIME | Check-out timestamp (UTC, nullable) |
| duration_mins | INTEGER | Duration of visit in minutes (calculated on checkout) |
| registered_at | DATETIME | Visit registration timestamp (UTC) |

# 9. Error Handling

All errors follow a consistent JSON format. The detail field contains a human-readable description of the error.

## Standard Error Response Format

```
{ "detail": "Incorrect email or password." }
```

## Validation Error Format (422)

```
{ "detail": [{ "loc": ["body", "email"], "msg": "value is not a valid email address",
"type": "value_error" }] }
```

## Common Error Scenarios

| Status | detail message | When it occurs |
|---|---|---|
| 401 | Incorrect email or password. | Login with wrong credentials |
| 401 | Could not validate credentials. | JWT token is expired or malformed |
| 403 | Access denied. Required role... | User's role is insufficient |
| 404 | Visit not found. | QR token or visit ID does not exist |
| 404 | Visitor not found. | Visitor ID does not exist |
| 409 | QR code has expired. | Visitor tried to use an expired QR |
| 409 | Visitor already checked out. | QR scanned after visitor already left |
| 422 | (Pydantic validation array) | Missing required field or wrong data type |
| 500 | (Internal server error) | Unexpected server-side exception |

# 10. Environment Configuration

The backend is configured via a .env file in the backend/ directory. Copy .env.example and fill in the values.

| Variable | Default | Description |
|---|---|---|
| DATABASE_URL | sqlite:///./vms.db | SQLAlchemy database connection string |
| SECRET_KEY | (required) | Random secret for JWT signing (min 32 chars recommended) |
| ACCESS_TOKEN_EXPIRE_MINUTES | 1440 | JWT token lifetime in minutes (1440 = 24 hours) |
| QR_EXPIRY_HOURS | 24 | Default QR code validity in hours |
| SMTP_ENABLED | false | Set to true to enable email delivery of QR codes |
| SMTP_HOST | smtp.gmail.com | SMTP server hostname |
| SMTP_PORT | 587 | SMTP port (587 for TLS, 465 for SSL) |
| SMTP_USER | (your email) | SMTP authentication username / email address |
| SMTP_PASSWORD | (app password) | SMTP authentication password (use App Password for Gmail) |
| SMTP_FROM | (your email) | From address shown in sent emails |
| APP_NAME | VisitorQR | Application name shown in emails and API title |
| APP_VERSION | 2.2 | API version string |

# 11. Quick Start Guide

## Step 1 — Start the Backend

```
cd vms_upgraded/backend
python -m venv venv
venv\Scripts\activate       # Windows
pip install -r requirements.txt
uvicorn main:app --reload
# API now running at http://localhost:8000
```

## Step 2 — Login

```
POST http://localhost:8000/api/auth/login
Content-Type: application/x-www-form-urlencoded
username=admin@company.com&password=your_password
```

## Step 3 — Register a Visitor

```
POST http://localhost:8000/api/visitors/register
Content-Type: application/json

{ "full_name": "Ramesh Kumar",
  "phone": "9876543210",
  "email": "ramesh@example.com",
  "purpose": "meeting",
  "branch_id": 1,
  "host_name": "Priya Sharma" }
```

## Step 4 — Scan QR at Entry

```
POST http://localhost:8000/api/visits/scan/{qr_token}
Authorization: Bearer <guard_jwt_token>
```

## Step 5 — View Dashboard

```
GET http://localhost:8000/api/admin/dashboard
Authorization: Bearer <admin_jwt_token>
```

## Interactive API Explorer

Open http://localhost:8000/docs in your browser for the full Swagger UI — you can test every endpoint directly from the browser without Postman.

*End of VisitorQR API Documentation — Version 2.2*

For support, refer to http://localhost:8000/docs or contact the system administrator.